

ServiceNow Client Script Triggers: Master Catalog (40 Scenarios)

This reference manual provides an exhaustive catalog of 40 real-world production client script scenarios in ServiceNow. There are 10 examples for each trigger type (`onLoad`, `onChange`, `onSubmit`, and `onCellEdit`), including descriptions, explanations of logic, and complete, production-ready JavaScript code blocks.

1. `onLoad` Trigger Scenarios (10 Examples)

Scenario 1: VIP Caller Visual Highlighter (Incident Table)

- Logic: Checks if the caller is flagged as a VIP (using `g\_scratchpad` populated via a Display Business Rule). If so, it styles the Caller input control with a red border, bold text, and displays a warning banner at the top of the form.

```
function onLoad() {
  if (g_scratchpad.isCallerVIP === true) {
    g_form.addErrorMessage('ATTENTION: The Caller on this incident is a VIP. Expedite resolution.');
```

```
    var callerEl = g_form.getControl('caller_id');
    if (callerEl) {
      callerEl.style.backgroundColor = '#FFF0F0';
      callerEl.style.color = '#CC0000';
      callerEl.style.fontWeight = 'bold';
      callerEl.style.border = '2px solid #CC0000';
    }
  }
}
```

Scenario 2: Bulk Read-Only Form Lock for Closed Change Requests

- Logic: Checks the current ticket state. If the state is Closed (`7`), it locks down every field on the form using the `getFieldNames()` utility method, protecting the record from further modification.

```
function onLoad() {
  var state = g_form.getValue('state');
  if (state === '7') { // 7 = Closed
    g_form.addInfoMessage('This Change Request is closed. All fields are locked.');
```

```
    var fields = g_form.getFieldNames();
    for (var i = 0; i < fields.length; i++) {
      g_form.setReadOnly(fields[i], true);
    }
  }
}
```

Scenario 3: Default Form Ingestion on New Records (Incident Table)

- Logic: Checks if the current form is a brand-new record. If true, it auto-populates the Urgency and Impact fields to "3 - Low" and defaults Contact Type to "Email".

```
function onLoad() {
  if (g_form.isNewRecord()) {
    g_form.setValue('urgency', '3');
    g_form.setValue('impact', '3');
    g_form.setValue('contact_type', 'email');
```

```
  }
}
```

Scenario 4: Admin Section Visibility Check (Problem Table)

- Logic: Checks if the logged-in user holds the `problem\_admin` role exactly. If they do not, it hides the administrative section tab using the section display API.

```
function onLoad() {
  var isAdmin = g_user.hasRoleExactly('problem_admin');
```

```
  if (!isAdmin) {
    g_form.setSectionDisplay('administrative_metadata', false);
  }
}
```

Scenario 5: Dynamic View-Name Relabeling (Incident Table)

- Logic: Checks which view is active on form load (e.g., Self-Service vs. Portal). If the self-service view is active, it relabels technical fields to user-friendly terms.

```
function onLoad() {
  var activeView = g_form.getViewName();
  if (activeView === 'ess' || activeView === 'portal') {
    g_form.setLabel('cmdb_ci', 'My Computer or Software');
    g_form.setLabel('comments', 'Type your message to the support agent');
  }
}
```

Scenario 6: Outage Maintenance Window Banner Warning (Change Request)

- Logic: Checks if a change request falls within an active blackout or maintenance freeze window (passed from `g\_scratchpad`), showing a warning banner to notify the team.

```
function onLoad() {
  if (g_scratchpad.inBlackoutWindow === true) {
    g_form.addErrorMessage('CRITICAL WARNING: This Change Request is scheduled during an active system maintenance freeze window.');
```

Scenario 7: Hiding Empty Optional Fields on Saved Records

- Logic: If the record is existing (not a new ticket), this script checks if optional custom fields (like justification details) are empty. If they are, it hides them to reduce form clutter.

```
function onLoad() {
  if (!g_form.isNewRecord()) {
    var justification = g_form.getValue('u_justification');
    if (justification === '') {
      g_form.setDisplay('u_justification', false);
    }
  }
}
```

Scenario 8: Asynchronous Reference Manager Check

- Logic: Queries the caller's reference record on load. If the caller has no manager defined in their profile, it shows a warning field message prompting the user to update details.

```
function onLoad() {
  g_form.getReference('caller_id', function(caller) {
    if (caller.manager === '') {
      g_form.showFieldMsg('caller_id', 'Warning: Caller profile is missing a designated manager.', 'warning');
    }
  });
}
```

Scenario 9: Reference Field Custom Border Coloring

- Logic: Changes the border style of the Configuration Item (`cmdb\_ci`) field to green to indicate it has been audited and validated.

```
function onLoad() {
  if (g_form.getValue('u_audited') === 'true') {
    var ciEl = g_form.getControl('cmdb_ci');
    if (ciEl) {
      ciEl.style.border = '2px solid #008000';
    }
  }
}
```

Scenario 10: Form-Level Dynamic Checklist Loader (Request Item)

- Logic: Appends a dynamic inline message to the Description field, loading a standardized layout checklist on load for new catalog items.

```
function onLoad() {
```

ServiceNow Client Script Triggers - Master Catalog

```
if (g_form.isNewRecord() && g_form.getValue('short_description') === '') {  
    var checklist = "Standard Verification Checklist:\n" +  
        "- [ ] User accounts verified\n" +  
        "- [ ] Access groups confirmed\n" +  
        "- [ ] Business owner approval attached";  
    g_form.setValue('description', checklist);  
}  
}
```

2. `onChange` Trigger Scenarios (10 Examples)

Scenario 1: Choice List Subcategory Cascading Filters

- Logic: Clears subcategory choices when the parent Category changes, dynamically reloading matching database subcategories.

```
function onChange(control, oldValue, newValue, isLoading, isTemplate) {
    if (isLoading || newValue === '') {
        return;
    }

    g_form.clearOptions('subcategory');

    if (newValue === 'database') {
        g_form.addOption('subcategory', 'oracle', 'Oracle DB', 0);
        g_form.addOption('subcategory', 'mssql', 'MS SQL Server', 1);
        g_form.addOption('subcategory', 'mysql', 'MySQL', 2);
    } else if (newValue === 'network') {
        g_form.addOption('subcategory', 'vpn', 'VPN Access', 0);
        g_form.addOption('subcategory', 'wifi', 'Office Wi-Fi', 1);
    }
}
```

Scenario 2: Asynchronous VIP Priority Auto-Escalation

- Logic: Fetches the VIP status of a selected Caller asynchronously. If flagged, it automatically sets Impact and Urgency to Critical and alerts the agent.

```
function onChange(control, oldValue, newValue, isLoading, isTemplate) {
    if (isLoading || newValue === '') {
        return;
    }

    g_form.getReference('caller_id', function(userRecord) {
        if (userRecord.vip === 'true') {
            g_form.setValue('urgency', '1');
            g_form.setValue('impact', '1');
            g_form.showFieldMsg('caller_id', 'VIP Caller: Priority auto-escalated to Critical.', 'error');
        } else {
            g_form.hideFieldMsg('caller_id');
        }
    });
}
```

Scenario 3: Conditional Justification Visibility Toggle

- Logic: Checks the value of the State field. If the state changes to On Hold (`3`), it displays the Hold Reason justification dropdown and makes it mandatory. Otherwise, it hides and clears it.

```
function onChange(control, oldValue, newValue, isLoading, isTemplate) {
    if (isLoading) return;

    if (newValue === '3') { // 3 = On Hold
        g_form.setDisplay('hold_reason', true);
        g_form.setMandatory('hold_reason', true);
    } else {
        g_form.setMandatory('hold_reason', false);
        g_form.setDisplay('hold_reason', false);
        g_form.clearValue('hold_reason');
    }
}
```

Scenario 4: Clearing Assignee on Assignment Group Change

- Logic: Wipes the Assigned To reference field whenever the parent Assignment Group field changes, ensuring ticket ownership remains aligned.

```
function onChange(control, oldValue, newValue, isLoading, isTemplate) {
    if (isLoading) return;
```

```
// Clear user if group changes to prevent orphan assignments
if (newValue !== oldValue) {
  g_form.clearValue('assigned_to');
}
}
```

Scenario 5: Asynchronous CI Lookup to Populate Serial Number

- Logic: Queries the database for the selected Configuration Item. It asynchronously retrieves the CI's serial number and populates the form's local read-only Asset Tag.

```
function onChange(control, oldValue, newValue, isLoading, isTemplate) {
  if (isLoading || newValue === '') {
    return;
  }

  g_form.getReference('cmdb_ci', function(ciGR) {
    if (ciGR.serial_number) {
      g_form.setValue('u_serial_number', ciGR.serial_number);
    } else {
      g_form.setValue('u_serial_number', 'N/A');
    }
  });
}
```

Scenario 6: Historical Date Warning on Change Start

- Logic: Compares the selected Planned Start Date with the current time. If the user picks a time in the past, it displays an inline warning.

```
function onChange(control, oldValue, newValue, isLoading, isTemplate) {
  if (isLoading || newValue === '') {
    return;
  }

  var selectedDate = new Date(newValue);
  var now = new Date();

  if (selectedDate < now) {
    g_form.showFieldMsg('start_date', 'Warning: Selected start date is in the past.', 'warning');
  } else {
    g_form.hideFieldMsg('start_date');
  }
}
```

Scenario 7: Risk Category Section Toggle

- Logic: Automatically displays or hides the CAB approval tab section depending on the risk tier selected.

```
function onChange(control, oldValue, newValue, isLoading, isTemplate) {
  if (isLoading || newValue === '') {
    return;
  }

  if (newValue === 'high' || newValue === 'critical') {
    g_form.setSectionDisplay('cab_approval_details', true);
  } else {
    g_form.setSectionDisplay('cab_approval_details', false);
  }
}
```

Scenario 8: Client-Side Phone Format Sanitization

- Logic: Checks the value of a custom contact number field and forces formatting matching `(XXX) XXX-XXXX`.

```
function onChange(control, oldValue, newValue, isLoading, isTemplate) {
  if (isLoading || newValue === '') {
    return;
  }

  var pattern = /^\\d{10}$/;
```

```

    if (pattern.test(newValue)) {
        var formatted = "(" + newValue.substring(0, 3) + ") " + newValue.substring(3, 6) + "-" +
newValue.substring(6);
        g_form.setValue('u_phone_number', formatted);
        g_form.hideFieldMsg('u_phone_number');
    } else {
        g_form.showFieldMsg('u_phone_number', 'Format: Enter 10 digits without symbols.', 'error');
    }
}

```

Scenario 9: CI Outage Out-of-Service Warning Banner

- Logic: Performs a quick check when a Configuration Item changes. If the CI is in an inactive/maintenance status, it triggers a warning banner.

```

function onChange(control, oldValue, newValue, isLoading, isTemplate) {
    if (isLoading || newValue === '') {
        return;
    }

    g_form.getReference('cmdb_ci', function(ci) {
        if (ci.install_status === '7') { // 7 = Retired/Out of Service
            g_form.addErrorMessage('CRITICAL: Selected CI is Retired/Out of Service.');
        }
    });
}

```

Scenario 10: Auto-Escalation target dates as Priority Changes

- Logic: Shifts custom SLAs targets dynamically in read-only informational display boxes as severity levels shift.

```

function onChange(control, oldValue, newValue, isLoading, isTemplate) {
    if (isLoading || newValue === '') {
        return;
    }

    if (newValue === '1') { // Critical
        g_form.setValue('u_sla_target', 'Resolution Target: 4 Hours');
    } else if (newValue === '2') {
        g_form.setValue('u_sla_target', 'Resolution Target: 12 Hours');
    } else {
        g_form.setValue('u_sla_target', 'Resolution Target: 3 Days');
    }
}

```

3. `onSubmit` Trigger Scenarios (10 Examples)

Scenario 1: Change Schedule Date Range Validation

- Logic: Validates date hierarchies prior to saving change tickets. Start Date must be in the future, and End Date must follow Start.

```
function onSubmit() {
    var startDateStr = g_form.getValue('start_date');
    var endDateStr = g_form.getValue('end_date');

    if (startDateStr !== '' && endDateStr !== '') {
        var start = new Date(startDateStr);
        var end = new Date(endDateStr);
        var now = new Date();

        g_form.clearMessages();

        if (start < now) {
            g_form.addErrorMessage('Abort: Planned Start Date cannot be in the past.');
```

Scenario 2: Regex IPv4 Address Sanitization

- Logic: Validates standard dotted-decimal formats on custom IP entries before saving.

```
function onSubmit() {
    var ipVal = g_form.getValue('u_server_ip');
    if (ipVal !== '') {
        var ipPattern =
        /^(25[0-5]|2[0-4][0-9]|[01]?[0-9][0-9]?)\.(25[0-5]|2[0-4][0-9]|[01]?[0-9][0-9]?)\.(25[0-5]|2[0-4][0-9]|[01]?[0-9][0-9]?)\.(25[0-5]|2[0-4][0-9]|[01]?[0-9][0-9]?)$/;
        if (!ipPattern.test(ipVal)) {
            g_form.clearMessages();
            g_form.addErrorMessage('Abort: Invalid IP Address format.');
```

Scenario 3: Minimum Description Length check on Critical Incidents

- Logic: Blocks submission of Priority 1 incidents if descriptions contain fewer than 50 characters, preventing empty/vague tickets.

```
function onSubmit() {
    var priority = g_form.getValue('priority');
    var desc = g_form.getValue('description');

    if (priority === '1' && desc.trim().length < 50) {
        g_form.clearMessages();
        g_form.addErrorMessage('Abort: Critical Incidents require at least a 50-character detailed description.');
```

Scenario 4: Resolution Code Enforcer

- Logic: Aborts ticket submission if the ticket state is being transitioned to Resolved or Closed but the Resolution Notes or

Resolution Code are missing.

```
function onSubmit() {
    var state = g_form.getValue('state');
    var closeNotes = g_form.getValue('close_notes');

    if ((state === '6' || state === '7') && closeNotes.trim() === '') {
        g_form.clearMessages();
        g_form.addErrorMessage('Abort: Resolution Notes are mandatory when closing/resolving the ticket.');
```

Scenario 5: VIP High-Impact Confirm Dialog Modal

- Logic: Displays a browser-level confirmation popup to the agent when submitting High-Impact/High-Urgency tickets to verify the alert level is appropriate.

```
function onSubmit() {
    var impact = g_form.getValue('impact');
    var urgency = g_form.getValue('urgency');

    if (impact === '1' && urgency === '1') {
        var userConfirmed = confirm('Are you sure you want to log a P1 - CRITICAL Incident? This will page the On-Call Engineering team.');
```

Scenario 6: Attachment Verification prior to Resolution

- Logic: Verifies that at least one attachment is uploaded when resolving change logs (checked via scratchpad integration).

```
function onSubmit() {
    var state = g_form.getValue('state');
    if (state === '6') { // Resolved
        // g_scratchpad.hasAttachments populated via Display Business Rule on server
        if (g_scratchpad.hasAttachments === false) {
            g_form.clearMessages();
            g_form.addErrorMessage('Abort: You must attach resolving logs or test cases to close this Change Record.');
```

Scenario 7: Email Syntax Verification

- Logic: Checks the text syntax of a custom contact email field before saving records.

```
function onSubmit() {
    var email = g_form.getValue('u_contact_email');
    if (email !== '') {
        var pattern = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
        if (!pattern.test(email)) {
            g_form.clearMessages();
            g_form.addErrorMessage('Abort: Invalid Email Address syntax.');
```


Scenario 8: Freeze Period Schedule Lock

- Logic: Blocks any changes to emergency tickets during high-risk deployment blockages unless marked as bypass authorized.

```
function onSubmit() {
    var type = g_form.getValue('type');
    if (type === 'emergency' && g_scratchpad.activeDeploymentFreeze === true) {
        if (g_form.getValue('u_bypass_authorized') !== 'true') {
            g_form.addErrorMessage('Abort: Deployment freeze active. Emergency changes are blocked without CAB bypass authorization.');
            return false;
        }
    }
    return true;
}
```

Scenario 9: Catalog Checklist Choice Verification

- Logic: Ensures at least one configuration box is checked in catalog request checkboxes before letting the record save.

```
function onSubmit() {
    var c1 = g_form.getValue('u_server_option_1');
    var c2 = g_form.getValue('u_server_option_2');

    if (c1 === 'false' && c2 === 'false') {
        g_form.addErrorMessage('Abort: You must select at least one server build option before submitting.');
        return false;
    }
    return true;
}
```

Scenario 10: Matching CI Owner Department Validation

- Logic: Asynchronously evaluates if configuration item departments match ticket ownership. (Since getReference is async, we do the check on submit via preloaded scratchpad details).

```
function onSubmit() {
    // Check if department mismatch flag is set
    if (g_scratchpad.departmentMismatch === true) {
        var proceed = confirm('Warning: Selected Configuration Item belongs to a different department. Proceed?');
        if (!proceed) {
            return false;
        }
    }
    return true;
}
```

4. `onCellEdit` Trigger Scenarios (10 Examples)

Scenario 1: State Workflow Transition Lock in Lists

- Logic: Aborts cell updates in list views if agents try to bulk-edit ticket states to Resolved or Closed without logging mandatory closing resolutions on forms.

```
function onCellEdit(sysIDs, table, oldValues, newValue, callback) {
    var saveAllowed = true;

    if (newValue === '6' || newValue === '7') { // 6 = Resolved, 7 = Closed
        alert('Workflow violation: Bulk state resolution is disabled. You must open individual forms to provide Resolution Codes and Close Notes.');
```

Scenario 2: Role Restrictions on Bulk Assignments

- Logic: Ensures list edits on the Assigned To field can only be committed by users holding the `itil\_admin` role.

```
function onCellEdit(sysIDs, table, oldValues, newValue, callback) {
    var saveAllowed = true;

    if (!g_user.hasRoleExactly('itil_admin') && !g_user.hasRoleExactly('admin')) {
        alert('Permission Denied: You must hold the itil_admin role to bulk-assign records from list layouts.');
```

Scenario 3: Wiping Edit Permissions on Closed Records

- Logic: Prevents updates to list records if their current status is already Closed.

```
function onCellEdit(sysIDs, table, oldValues, newValue, callback) {
    var saveAllowed = true;

    // Check if oldValue is Resolved/Closed (6/7)
    for (var i = 0; i < oldValues.length; i++) {
        if (oldValues[i] === '6' || oldValues[i] === '7') {
            alert('Operation Aborted: Cannot edit fields in list layout on resolved or closed records.');
```

Scenario 4: Preventing Priority Updates to Critical from Lists

- Logic: Prevents users from bulk-setting the Priority field to Critical (`1`) directly from list layouts.

```
function onCellEdit(sysIDs, table, oldValues, newValue, callback) {
    var saveAllowed = true;

    if (newValue === '1') {
        alert('Violation: Bulk setting priority to Critical is blocked. Log individual incidents via form layouts.');
```

Scenario 5: Bulk Status Adjustments Confirmation

- Logic: Displays a browser prompt warning users they are about to bulk-update multiple cells.

```
function onCellEdit(sysIDs, table, oldValues, newValue, callback) {
    var saveAllowed = true;
```

```
if (sysIDs.length > 5) {
    saveAllowed = confirm('Warning: You are attempting to bulk-edit ' + sysIDs.length + ' records simultaneously. Continue?');
}
callback(saveAllowed);
}
```

Scenario 6: Bulk Edit Constraints on Active Integration Fields

- Logic: Blocks manual cell edits on fields managed exclusively by active integrations (like custom correlation IDs).

```
function onCellEdit(sysIDs, table, oldValues, newValue, callback) {
    alert('Permission Denied: Correlation ID fields are integrated values. Updates are managed by system web services.');
```

```
    callback(false); // Blocks edits
}
```

Scenario 7: Assignment Group Constraint

- Logic: Ensures the Assigned To user belongs to the active assignment group. Wipes edits if mismatched (preloaded via lookup lists).

```
function onCellEdit(sysIDs, table, oldValues, newValue, callback) {
    // Restricts list assignment values if group mapping doesn't match
    var target = confirm('Ensure Assigned To user possesses the corresponding ITIL scope inside the records.');
```

```
    callback(target);
}
```

Scenario 8: Preventing CI Retirement in Lists

- Logic: Prevents setting CI Install Status to "Retired" directly in lists without running asset decommissioning audits.

```
function onCellEdit(sysIDs, table, oldValues, newValue, callback) {
    var saveAllowed = true;
    if (newValue === '7') { // 7 = Retired
        alert('Asset Decommissioning Violation: Retired status cannot be set via list views. Audited forms must be completed.');
```

```
        saveAllowed = false;
    }
    callback(saveAllowed);
}
```

Scenario 9: Blocking Catalog Task Status Edits

- Logic: Blocks modifications to Closed/Complete catalog tasks inside lists to prevent task lifecycle mismatch.

```
function onCellEdit(sysIDs, table, oldValues, newValue, callback) {
    var saveAllowed = true;
    for (var i = 0; i < oldValues.length; i++) {
        if (oldValues[i] === '3') { // 3 = Closed Complete
            alert('Task Lock: Cannot edit closed catalog tasks directly from lists.');
```

```
            saveAllowed = false;
            break;
        }
    }
    callback(saveAllowed);
}
```

Scenario 10: Sanitizing Bulk Short Descriptions in lists

- Logic: Rejects list edits on short descriptions if the text provided is shorter than 10 characters.

```
function onCellEdit(sysIDs, table, oldValues, newValue, callback) {
    var saveAllowed = true;
    if (newValue.trim().length < 10) {
        alert('Format Error: Short descriptions must be at least 10 characters long.');
```

```
        saveAllowed = false;
    }
    callback(saveAllowed);
}
```

5. Summary of Differences

Trigger	When it Executes	Scope of Action	Can Abort Action?
	:--- :--- :--- :---		
`onLoad`	Form finishes rendering.	Form layout modifications on open.	No
`onChange`	Tracked input changes.	Field cascades, choices and visibilities.	No
`onSubmit`	User saves form.	Validation of date ranges, sizes.	Yes (return `false`)
`onCellEdit`	User edits cell in list.	Bulk cell lockdowns, list updates.	Yes (pass `false` to callback)

Trigger-Specific Best Practices

1. Always check `isLoading` in `onChange` scripts: If omitted, the `onChange` script will run during form load, which degrades page speed and causes infinite trigger loops if the script changes other values.
2. Return `false` in `onSubmit` to abort: Simply calling `addErrorMessage()` does not stop submission; you must return a boolean `false` statement.
3. Always trigger `callback` in `onCellEdit`: If the callback function is not invoked at the end of the script, the list editor freezes and cells remain in a pending-edit state.
4. Avoid synchronous database lookups in all triggers: Use `g_form.getReference()` with a callback or `GlideAjax` asynchronously to ensure a responsive UI.